CMPE 300: Analysis of Algorithms

Project 1

Kübra Uykun
kubra.uykun@bogazici.edu.tr

Mert Özüstün
mert.ozustun@std.bogazici.edu.tr

Osman Melih Bal
osman.bal@std.bogazici.edu.tr

Strict Deadline: Friday 13 November 2026 23:59

1 Introduction

This project is designed to assess your understanding of the core concepts in algorithm analysis. You will be given the pseudocode of an algorithm. Your task is to thoroughly analyze this algorithm and determine its basic operation(s), calculate its best, worst, and average-case time complexities, implement it in code, verify your exact counts experimentally, execute it and compare the theoretical results with the actual execution times.

Every group works on its own variant of the algorithm (see Section 2.1). Results belonging to another variant will receive no points.

2 Details of the Algorithm

Consider the following algorithm which takes a list `arr[0:n-1]` as input. Each element in the list is 'a', 'b', or 'c'; i.e., $arr[i] \in \{\text{'a'}, \text{'b'}, \text{'c'}\}$, $0 \leq i \leq n-1$. The elements are independent; for each i, the probability that $arr[i] =$ 'a' is p_a, the probability that $arr[i] =$ 'b' is p_b, and the probability that $arr[i] =$ 'c' is p_c ($p_a + p_b + p_c = 1$). The values of p_a, p_b, p_c and the integer $d \geq 2$ used in line 7 depend on your variant.

Algorithm 1 Example Algorithm

input: $arr[0 : n-1]$ (a list of size n), integer d (from your variant)
output: res (an integer)

```
1:  res ← 0;  s ← 0
2:  for i ← 0 to n − 1 do
3:      if arr[i] = 'a' then                                  ▷ (1)
4:          s ← s + 1                                         ▷ (3)
5:          k ← n
6:          while k > 0 do
7:              k ← ⌊k/d⌋                                     ▷ (4)
8:      else if arr[i] = 'b' then                             ▷ (1)
9:          for q ← 0 to s do                                 ▷ (2)
10:             res ← res + q                                 ▷ (3)
11:             for r ← 1 to n do
12:                 res ← res + r                             ▷ (4)
13:     else                                                  ▷ arr[i] = 'c'
14:         s ← 0                                             ▷ (3)
15:         for t ← i to n − 1 do                             ▷ (2)
16:             res ← res + t                                 ▷ (4)
17: return res
```

Note that the variable s is *not* reset at the beginning of each iteration of the outer loop: the work done for $arr[i]$ may depend on the elements that come before it.

2.1 Your Variant

Let No_1 and No_2 be the student numbers of the two group members. Your variant key is

$$key = (No_1 + No_2) \bmod 8.$$

key	p_a	p_b	p_c	d
0	1/2	1/4	1/4	2
1	1/2	1/4	1/4	3
2	1/3	1/3	1/3	2
3	1/3	1/3	1/3	3
4	2/5	2/5	1/5	2
5	2/5	2/5	1/5	3
6	1/6	1/2	1/3	2
7	1/6	1/2	1/3	3

Write your key and parameters in your report, using the Variant table of the provided template. Your final formulas must be given for your own parameters (you may first derive them for general p_a, p_b, p_c, d and then substitute).

3 Theoretical Analysis

Consider each of the following four cases separately (one by one):

1. Basic operations are the two comparisons marked as (1)
2. Basic operations are the two loop increments marked as (2)
3. Basic operations are the three assignments marked as (3)
4. Basic operations are the three assignments marked as (4)

For each case:

a) Analyze $B(n)$

b) Analyze $W(n)$

c) Analyze $A(n)$

Your analyses must be exact, otherwise no points will be given. That is, for each analysis, first find the "exact" number of basic operations as a closed-form expression of n (floors, ceilings, powers such as c^n, and $\log_d$ are allowed; summations are not) and then convert it to asymptotic notation. (Note that best-case input and worst-case input may be different for different cases.) So, you will obtain 12 analysis results. The final result of each of these 12 analyses (the exact count and the asymptotic notation) must also be entered into Table 1 of the provided template. Table 1 only collects the final results; it does not replace the derivations and proofs required in this section.

Additional requirements:

- For every $B(n)$ and $W(n)$, explicitly state a best-case (worst-case) input of size n and *prove* that no other input of size n executes fewer (more) basic operations. Showing that an input is "bad" ("good") is not enough; you must argue why it is the worst (best).
- For every $A(n)$, state clearly the random variables you use and how you compute their expectations. Pay attention to the variable s.

4 Real Execution

Identify the "correct" basic operation(s), i.e., the operation that characterizes this algorithm. Comment on the consistency of the theoretical analysis and the identification of the basic operation set, and use the best-case and worst-case inputs of that operation in this section.

Code this algorithm in a programming language you wish. Execute it on a computer with 15 different input sizes $n \in \{1, 5, 10, 20, 40, 50, 60, 80, 100, 150, 200, 250, 300, 350, 400\}$, for three types of inputs (best-case input, worst-case input, average-case input). For each, record the actual execution time (in seconds). At the end, you should have 45 different time records. Then, you will fill Table 2 of the provided template with the results you get, and state the programming language and the machine you used. Also, your code should print out the results. For more information, see Section 7.

As an average-case input, you can generate a random array `arr[0:n-1]` formed of 'a's, 'b's and 'c's according to the probability distribution of your variant. Note that in order to observe the average behavior, for each input size, you must execute the algorithm with random inputs several times and take the average. For this project, run the algorithm at least 10 times for taking the average.

5 Exact Count Verification

Your code must also be able to *count* the basic operations. Add one counter for each of the four cases (1)–(4) to your implementation (the counters must not be used while measuring execution times). Then:

1. For each case (1)–(4) and for $n \in \{1, 5, 10, 20, 40, 100\}$, run your algorithm on the best-case and worst-case input of that case and fill Table 3 of the provided template with the counted values and the values given by your $B(n)$ and $W(n)$ formulas. They are expected to be equal.

2. For each case (1)–(4) and for $n \in \{10, 40, 100\}$, run your algorithm on at least 1000 random inputs (generated according to the probability distribution of your variant), and report the mean of the counted values together with your $A(n)$ value in Table 4 of the provided template. Comment on the difference.

If a counted value does not match your formula, report it anyway: a correct mismatch report with a correct explanation is worth more than a fabricated table.

6 Comparison of Theoretical Analysis and Real Behavior

Compare the theoretical results and the actual execution times using graphs. Use graphs where the x-axis denotes the input size and the y-axis denotes the complexity (for the theoretical analysis) and time (for the actual execution). Your comparisons must include all the theoretical and actual analyses you derived above. For each of the 12 theoretical analyses, comment on the result of the analysis by comparing it with the related actual execution time. In particular, compare the asymptotic growth of the average case with that of the worst case and explain the reason using the pseudocode. No points will be given if any comment is missing or the comments are not satisfactory.

7 Code

You will also submit your code. You are free to select any language. (Python is preferred.) Yet, please provide a ReadMe file that simply explains how to run your code. Your program must take the two student numbers as command line arguments and print your variant in the first line as:

```
Variant: <key> (p_a=<p_a>, p_b=<p_b>, p_c=<p_c>, d=<d>)
```

Then you should print the time elapsed for best, worst and average cases, and for 15 different data sizes n as:

```
Case: <case> Size: <size> Elapsed Time (s): <time>
```

where,

- `<case>` denotes "best", "worst" or "average"
- `<size>` denotes the data size

- `<time>` denotes the time (in seconds)

For each average-case result, you should also print the execution times of all the runs from which you took the average.

Count mode. When it is called as

```
python3 StudentNumber.py <No1> <No2> --count <string>
```

where `<string>` is an input such as `aabcbba`, your program must run the algorithm once on that input and print only the variant line and

```
Count(1): <c1> Count(2): <c2> Count(3): <c3> Count(4): <c4>
```

Your code will be tested with hidden inputs in this mode.

8 Submission

- This project is designed to be completed by a pair of two students.
- The answers of all the questions must be collected into a single report. Four table templates in LaTeX are provided to you: Table 1 (Theoretical Analysis Results, together with the Variant table for your key and parameters), Table 2 (Real Execution Times), Table 3 (Exact Count Verification, best and worst case) and Table 4 (Exact Count Verification, average case). You are expected to include these tables in your report and fill them in. The structure of the rest of the report is up to you; follow the sections of this description.
- **Important:** The tables are provided only to display your final data in a uniform way. Filling them in does not complete the project. You must still write your full report: the exact derivations of $B(n)$, $W(n)$ and $A(n)$ for all four cases, the explicit best-case and worst-case inputs together with the proofs that no other input executes fewer (more) basic operations, the identification of the basic operation, the exact count verification, the graphs, and all the comments requested in this description. A report that consists only of filled tables will not receive points.
- Prepare the report on a computer, not handwritten. Since the tables are provided in LaTeX, using LaTeX is recommended.
- In addition to the project files, each student in a group must submit a document stating explicitly the parts of the project she/he has worked on. The document must begin with the line "I worked on the following parts in this project:" and all the parts the student has worked on must be explained very clearly and in detail. Do not write general comments such as "we worked on the project together", etc. Write what you have done in the project explicitly. If both students write the same or similar explanations, their projects will not be graded.
- You should also submit the program code.
- Each student in the group should upload a single zip file on Moodle, which consists of the pdf file of the answers, a single file that is the program code, the ReadMe file and the document that explains the parts of the project you worked on. Name the files as follows:

  - Answers: `StudentNumber.pdf`
  - Program code: `StudentNumber.xxx` (xxx: depending on the language)
  - Document: `StudentNumber_WorkDone.pdf`
  - Zip file that includes these files: `StudentNumber.zip`
- Deadline is Friday, 13 November 2026, 23:59. Deadline is strict.
- No late submission will be accepted.